Unit 2: Searching and Sorting

2.1 Searching

Searching is the process of finding a particular element, called a **key**, from a collection of data.

For example, consider:

Array = 20 15 40 10 30

Key = 10

The searching process checks the elements of the array to determine whether 10 is present.

If the element is present → **Search Successful and display key elements position in array**

If the element is not present → **Search Unsuccessful**

Applications of Searching

Searching is used in:

- Finding a student by roll number
- Finding an employee by employee ID
- Finding a product by product ID
- Finding a record in a database
- Finding a word in a dictionary
- Searching files in a computer

Common Searching Techniques

1. **Linear Search**
2. **Binary Search**

2.1.1 Linear Search

Linear Search is a searching technique in which each element of the array is checked **sequentially from the beginning until the required element is found or the array ends**.

It is also called **Sequential Search**.

Example

Consider:

Array: 20 15 40 10 30
Index: 0 1 2 3 4

Key = 10

Searching process:

20 → 15 → 40 → 10
↑
Found

The key 10 is found at **index 3**.

C Program: Linear Search

```
#include <stdio.h>
int main()
{
   int n, key, i, found = 0;

   printf("Enter number of elements: ");
   scanf("%d", &n);

   int a[n];

   printf("Enter elements:\n");
   for(i = 0; i < n; i++)
   {
      scanf("%d", &a[i]);
   }

   printf("Enter element to search: ");
   scanf("%d", &key);

   for(i = 0; i < n; i++)
   {
      if(a[i] == key)
      {
         printf("Element found at index %d", i);
         found = 1;
         break;
      }
   }

   if(found == 0)
```

```
    {
        printf("Element not found");
    }

    return 0;
}
```

Time Complexity of Linear Search

Case	Time Complexity
Best Case	**O(1)**
Average Case	**O(n)**
Worst Case	**O(n)**

2.1.2 Binary Search

Binary Search is an efficient searching technique that repeatedly divides a **sorted array into two halves** to find the required element.

Important Condition

> **Binary Search can be applied only when the data is sorted.**

For example:

Sorted Array:

10 20 30 40 50 60 70

Working of Binary Search

Consider:

Array:

```
10  20  30  40  50  60  70
 ↑           ↑           ↑
low         mid         high
```

Suppose:

Key = 60

Step 1

Middle element:

mid = (low + high) / 2

Middle element is 40.

Compare:

60 > 40

Therefore, search in the **right half**.

10 20 30 40 | 50 60 70
↑
Search here

Step 2

Middle element of the remaining portion is 60.

60 == 60

Therefore:

Element Found

C Program: Binary Search

```
#include <stdio.h>

int main()
{
   int n, key;
   int low, high, mid;
   int found = 0;

   printf("Enter number of elements: ");
   scanf("%d", &n);

   int a[n];

   printf("Enter sorted elements:\n");
  for(int i = 0; i < n; i++)
   {
      scanf("%d", &a[i]);
   }
```

```
    printf("Enter element to search: ");
    scanf("%d", &key);

    low = 0;
    high = n - 1;

    while(low <= high)
    {
        mid = (low + high) / 2;

        if(a[mid] == key)
        {
            printf("Element found at index %d", mid);
            found = 1;
            break;
        }
        else if(key < a[mid])
        {
            high = mid - 1;
        }
        else
        {
            low = mid + 1;
        }
    }

    if(found == 0)
    {
        printf("Element not found");
    }

    return 0;
}
```

Time Complexity of Binary Search

Case	Time Complexity
Best Case	**O(1)**
Average Case	**O(log n)**
Worst Case	**O(log n)**

Linear Search vs Binary Search

Feature	Linear Search	Binary Search
Data requirement	Sorted/Unsorted	**Sorted**
Method	Sequential checking	Divide into halves
Best Case	O(1)	O(1)
Worst Case	O(n)	**O(log n)**
Suitable for	Small/unsorted data	Large/sorted data
Implementation	Easy	Moderate

2.2 Sorting

Introduction to Sorting

Sorting is the process of arranging data in a particular order.

The two common orders are:

Ascending Order
10 20 30 40 50

Descending Order
50 40 30 20 10

Why is Sorting Required?

Sorting makes data:

- Easier to search
- Easier to analyze
- Easier to understand
- Easier to process
- Useful for finding minimum and maximum
- Useful for efficient searching such as binary search

Sorting Techniques

Your syllabus includes:

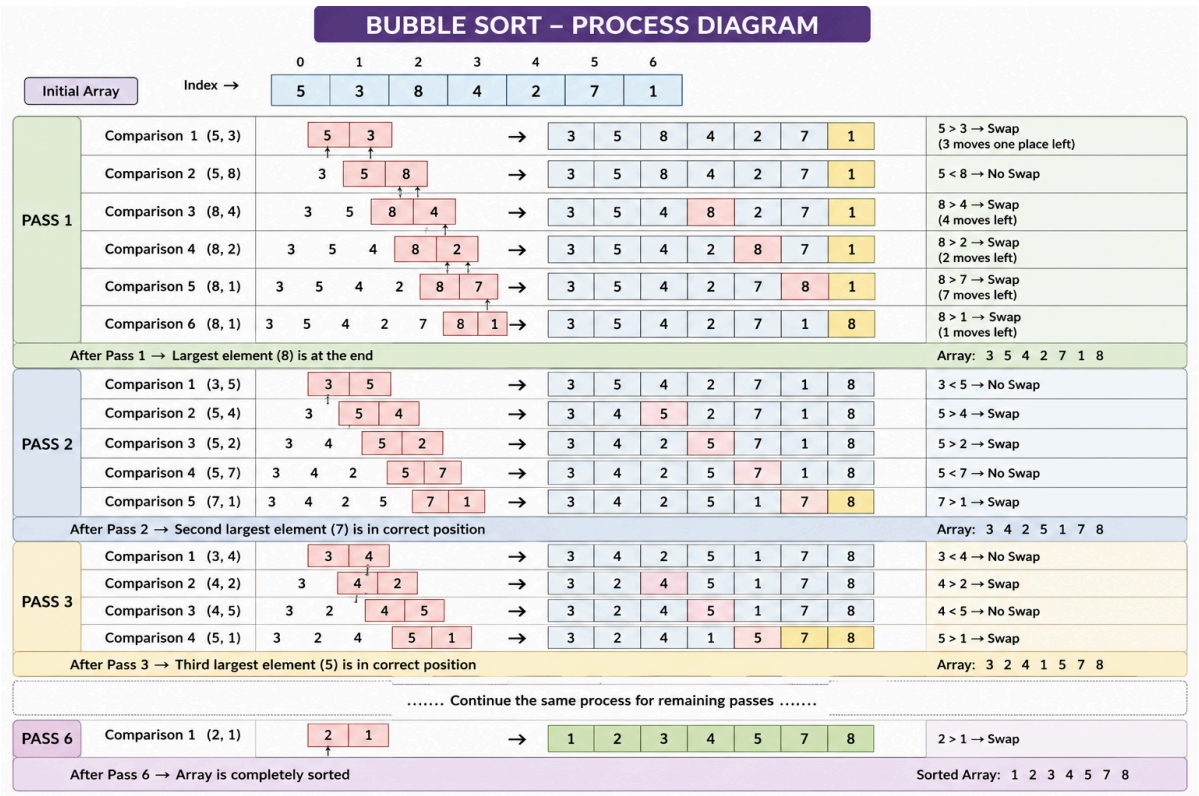
1. **Bubble Sort**
2. **Selection Sort**
3. **Insertion Sort**
4. **Quick Sort**
5. **Merge Sort**

2.2.1 Bubble Sort

Bubble Sort repeatedly compares **adjacent elements** and swaps them if they are in the wrong order.

The largest element gradually moves toward the end of the array after each pass.

This movement is similar to a bubble moving upward, hence the name **Bubble Sort**.



C Program: Bubble Sort

```
#include <stdio.h>

int main()
{
   int n, temp;
```

```
    printf("Enter number of elements: ");
    scanf("%d", &n);

    int a[n];

    printf("Enter elements:\n");
    for(int i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

    for(int i = 0; i < n - 1; i++)
    {
        for(int j = 0; j < n - i - 1; j++)
        {
            if(a[j] > a[j + 1])
            {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }

    printf("Sorted array:\n");

    for(int i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }

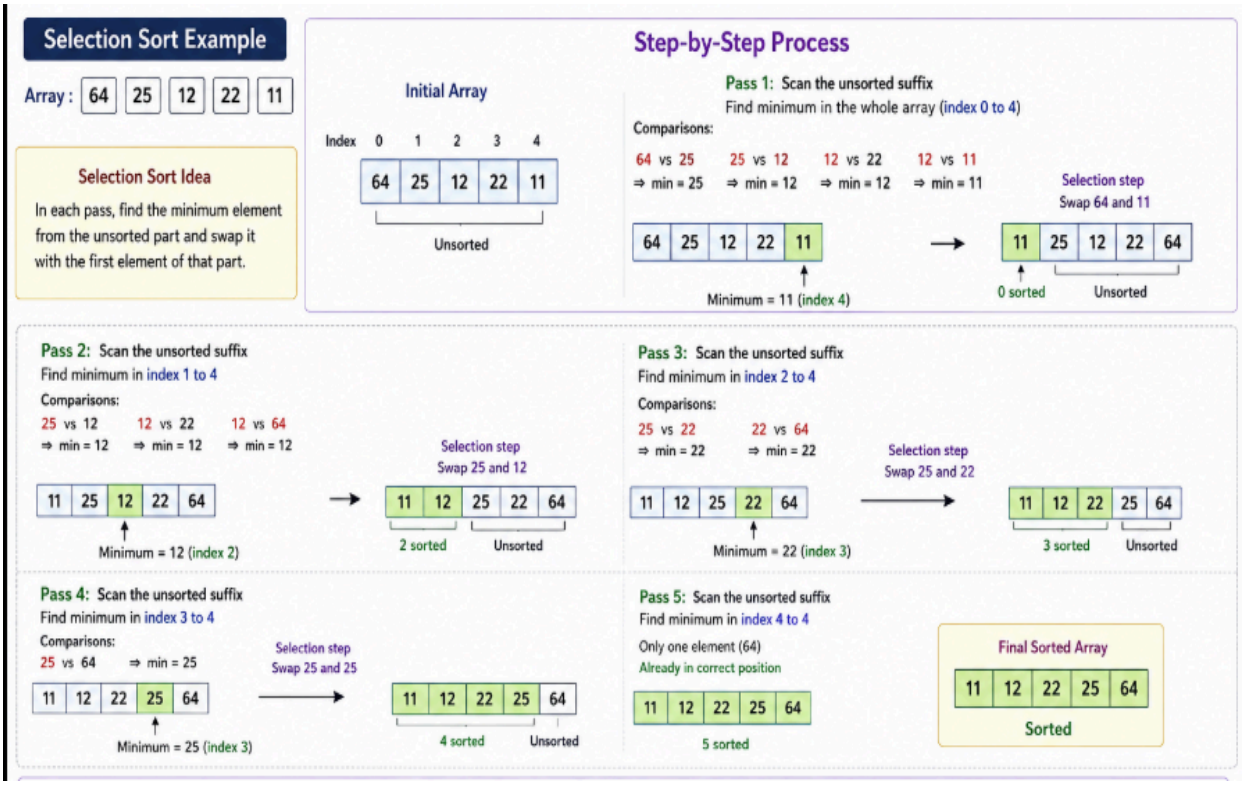
    return 0;
}
```

Complexity

Case	Complexity
Best Case*	O(n)
Average Case	O(n²)
Worst Case	O(n²)
Space	O(1)

2.2.2 Selection Sort

Selection Sort repeatedly selects the **smallest element** from the unsorted portion and places it at the correct position.



C Program: Selection Sort

```
#include <stdio.h>

int main()
{
   int n, min, temp;

   printf("Enter number of elements: ");
   scanf("%d", &n);

   int a[n];

   printf("Enter elements:\n");
   for(int i = 0; i < n; i++)
   {
      scanf("%d", &a[i]);
   }

   for(int i = 0; i < n - 1; i++)
   {
      min = i;
```

```
        for(int j = i + 1; j < n; j++)
        {
            if(a[j] < a[min])
            {
                min = j;
            }
        }

        temp = a[i];
        a[i] = a[min];
        a[min] = temp;
    }

    printf("Sorted array:\n");

    for(int i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }

    return 0;
}
```

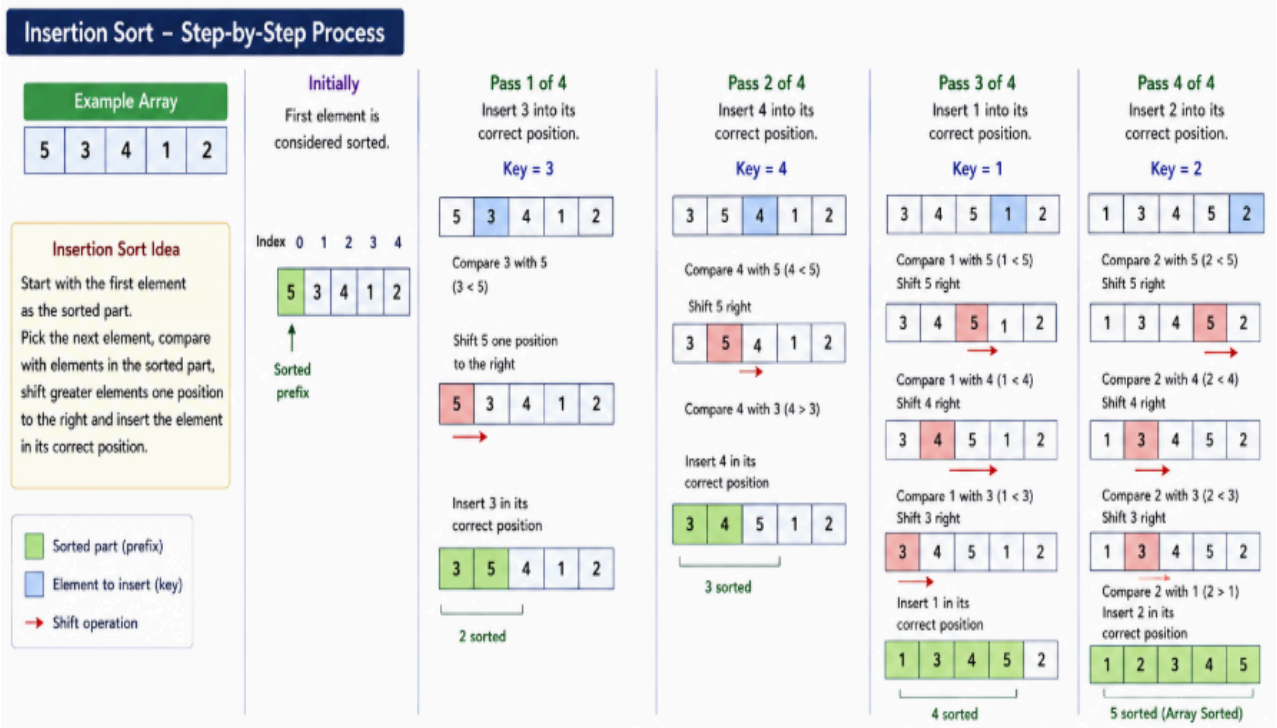
Complexity

Case	Complexity
Best Case	$O(n^2)$
Average Case	$O(n^2)$
Worst Case	$O(n^2)$
Space	O(1)

2.2.3 Insertion Sort

Insertion Sort builds the sorted array **one element at a time**.

It works similarly to arranging playing cards in your hand.



C Program: Insertion Sort

```
#include <stdio.h>

int main()
{
   int n, key, j;

   printf("Enter number of elements: ");
   scanf("%d", &n);

   int a[n];

   printf("Enter elements:\n");
   for(int i = 0; i < n; i++)
   {
      scanf("%d", &a[i]);
   }

   for(int i = 1; i < n; i++)
   {
      key = a[i];
      j = i - 1;

      while(j >= 0 && a[j] > key)
      {
```

```
            a[j + 1] = a[j];
            j--;
        }

        a[j + 1] = key;
    }

    printf("Sorted array:\n");

    for(int i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }

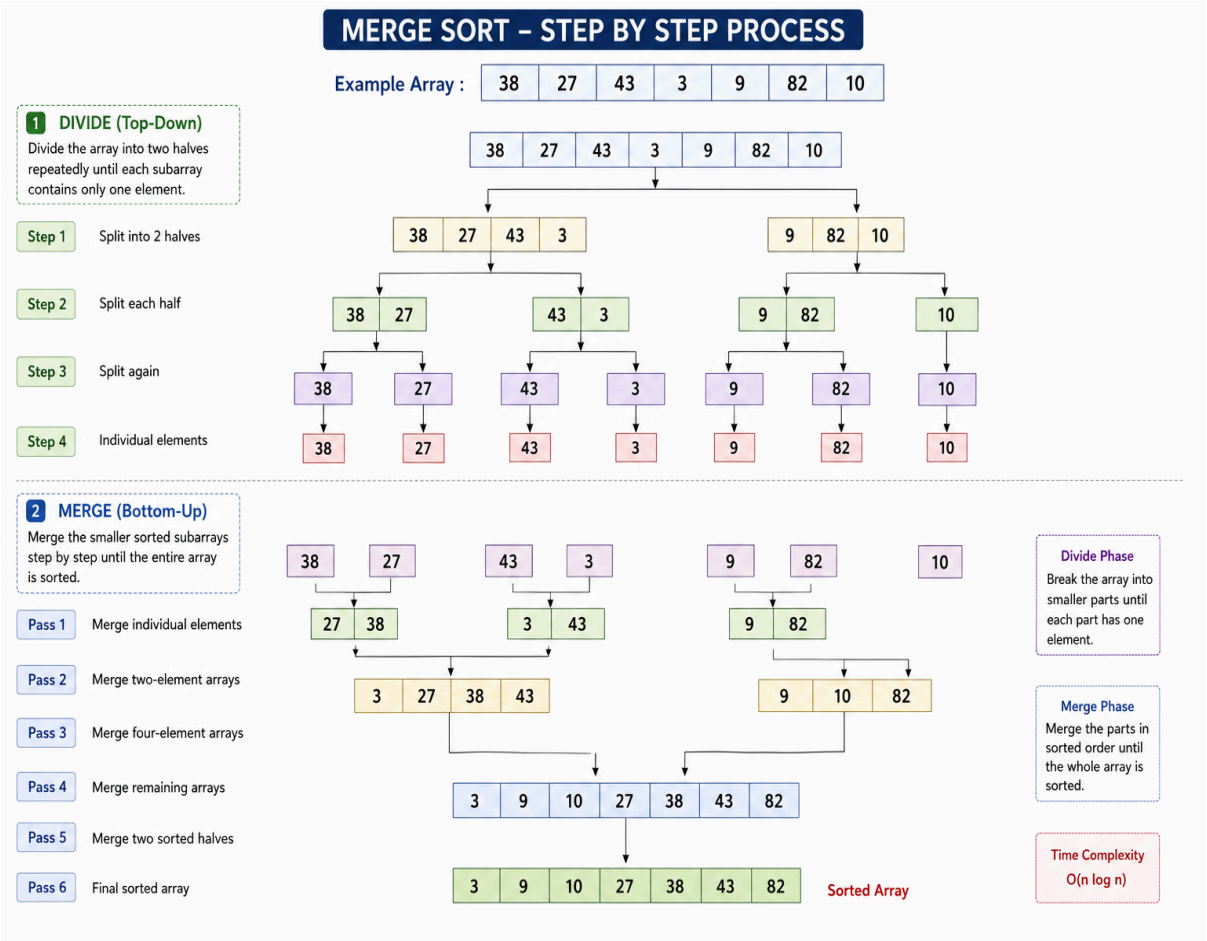
    return 0;
}
```

Complexity

Case	Complexity
Best Case	**O(n)**
Average Case	$O(n^2)$
Worst Case	$O(n^2)$
Space	O(1)

2.2.4 Merge Sort

Merge Sort is a **divide-and-conquer** sorting algorithm that divides the array into smaller subarrays, sorts them, and then merges the sorted subarrays.



C Program: Merge Sort

```
#include <stdio.h>

void merge(int a[], int low, int mid, int high)
{
   int i = low;
   int j = mid + 1;
   int k = 0;

   int temp[high - low + 1];

   while(i <= mid && j <= high)
   {
      if(a[i] <= a[j])
      {
         temp[k] = a[i];
         i++;
      }
      else
      {
```

```
            temp[k] = a[j];
            j++;
        }

        k++;
    }

    while(i <= mid)
    {
        temp[k] = a[i];
        i++;
        k++;
    }

    while(j <= high)
    {
        temp[k] = a[j];
        j++;
        k++;
    }

    for(i = low, k = 0; i <= high; i++, k++)
    {
        a[i] = temp[k];
    }
}

void mergeSort(int a[], int low, int high)
{
    if(low < high)
    {
        int mid = (low + high) / 2;

        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);

        merge(a, low, mid, high);
    }
}

int main()
{
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int a[n];
```

```
    printf("Enter elements:\n");
    for(int i = 0; i < n; i++)
    {
        scanf("%d", &a[i]);
    }

    mergeSort(a, 0, n - 1);

    printf("Sorted array:\n");

    for(int i = 0; i < n; i++)
    {
        printf("%d ", a[i]);
    }

    return 0;
}
```

Complexity

Case	Complexity
Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	O(n log n)
Space	O(n)

Key Point

Merge Sort guarantees **O(n log n)** time complexity, but it requires additional memory for merging.

2.2.5 Quick Sort

Quick Sort is a **divide-and-conquer** sorting algorithm.

It selects an element called a **pivot** and partitions the array into two parts:

- Elements smaller than pivot
- Elements greater than pivot

Then Quick Sort is recursively applied to both parts.

Important Terms

Pivot

The element selected for partitioning the array.

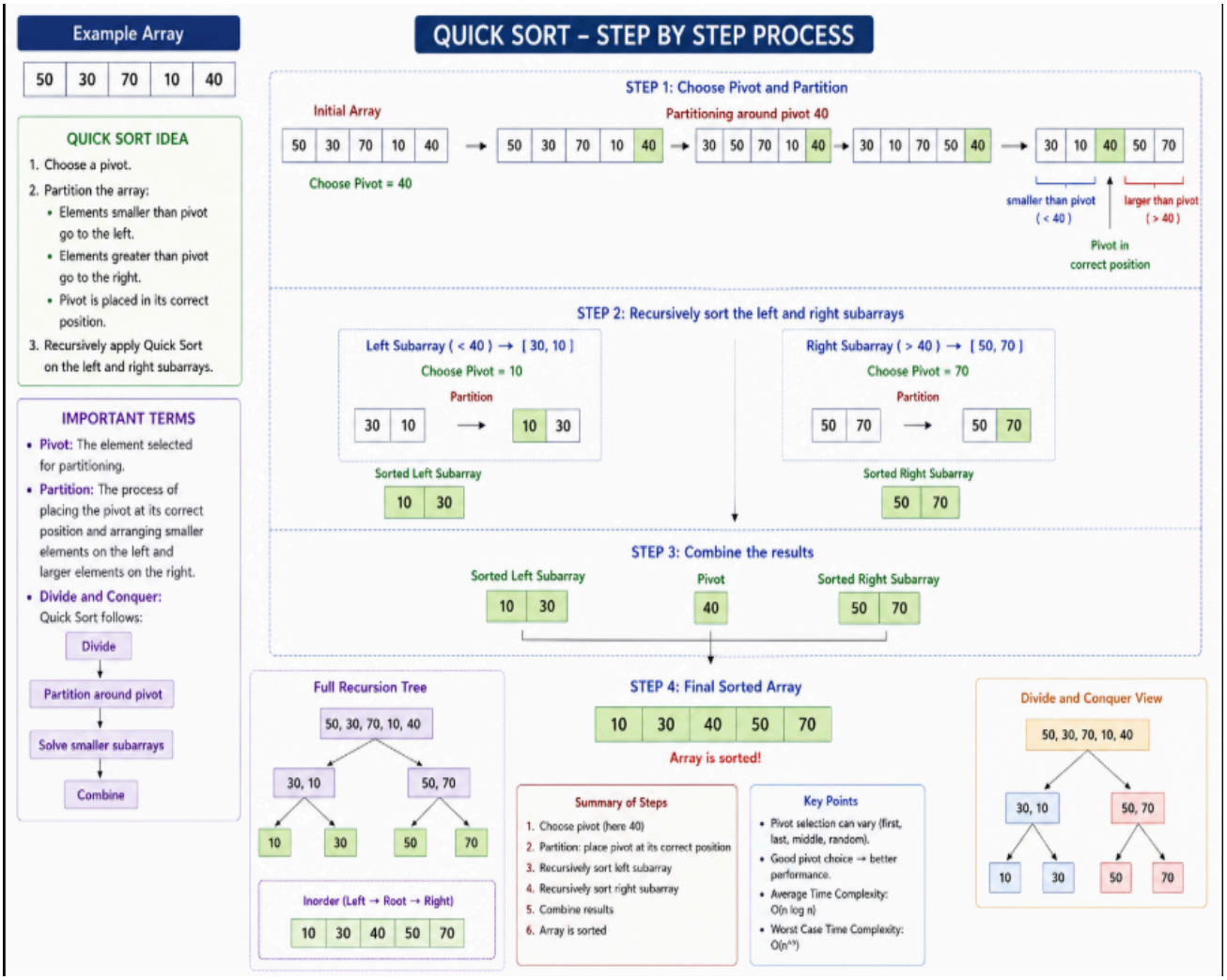
Partition

The process of placing the pivot at its correct position and arranging smaller elements on one side and larger elements on the other side.

Divide and Conquer

Quick Sort follows:

Divide
↓
Partition around pivot
↓
Solve smaller subarrays
↓
Combine



Complexity of Quick Sort

Case	Complexity
Best Case	O(n log n)
Average Case	O(n log n)
Worst Case	$O(n^2)$
Space	O(log n) average recursion stack

Comparison of Sorting Algorithms

Algorithm	Best	Average	Worst	Extra Space
Bubble Sort	O(n)*	O(n²)	O(n²)	O(1)
Selection Sort	O(n²)	O(n²)	O(n²)	O(1)
Insertion Sort	O(n)	O(n²)	O(n²)	O(1)
Quick Sort	O(n log n)	O(n log n)	O(n²)	O(log n)**
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)

QUESTIONS

Q1. Answer the following

1. Define Pivot in Quick Sort.
2. What is Binary Search?
3. List any five sorting techniques covered in the syllabus.
4. Define Searching.

Q2. Answer the following

1. Explain the difference between Linear Search and Binary Search.
2. Explain why Binary Search requires a sorted array.
3. Explain the working principle of selection Sort.
4. Explain the working principle of bubble sort..
5. Explain the Divide and Conquer approach used in Merge Sort.

Q3. Solve/Apply the following

1. Apply **Linear Search** to find 40 in the array Also, write a C program to implement linear Search.
   20, 15, 40, 10, 30

2. Apply **Binary Search** to find 60 in:Also, write a C program to implement Binary Search.
   10, 20, 30, 40, 50, 60, 70

3. Apply **Bubble Sort** to sort the following array in ascending order:Also, write a C program to implement Bubble sort.
   64, 25, 12, 22, 11
4. Apply **Insertion Sort** to sort: Also, write a C program to implement Insertion sort.
   5, 3, 4, 1, 2
5. Apply **selection Sort** to the following array in ascending order. write a C program to implement selection sort.
   50, 30, 70, 10, 40

Q4. Analyze the following

1. Analyze the difference between **Linear Search and Binary Search** with respect to:
   - Data requirement
   - Searching method
   - Best-case complexity
   - Worst-case complexity
   - Suitable application
2. Analyze **Merge Sort and Quick Sort** as divide-and-conquer algorithms. Compare their:
   - Division/partitioning process
   - Recursive processing
   - Best-case complexity
   - Worst-case complexity
   - Extra space requirement

3. Analyze the following **Selection Sort** array and identify the minimum element selected in each pass:

64, 25, 12, 22, 11